

ELF header

All ELF object files start with the ELF header. It is in binary but has the following data structure format:

```
# define EI_NIDENT 16

typedef struct {
    unsigned char
    e_ident[EI_NIDENT];
    Elf64_Half    e_type;
    Elf64_Half    e_machine;
    Elf64_Word    e_version;
    Elf64_Addr    e_entry;
    Elf64_Off     e_phoff;
    Elf64_Off     e_shoff;
    Elf64_Word    e_flags;
    Elf64_Half    e_ehsize;
    Elf64_Half    e_phentsize;
    Elf64_Half    e_phnum;
    Elf64_Half    e_shentsize;
    Elf64_Half    e_shnum;
    Elf64_Half    e_shstrndx;
} Elf64_Ehdr;
```

e_ident: This variable marks the file as an ELF object file and provides information about the object file.

e_type: This member determines the object file type.

e_machine: This member specifies the architecture for which this ELF is compiled.

e_version: This member provides information about the version of the ELF object file.

e_entry: This is the entry point or the start address of the program.

e_phoff: This member provides offset from the ELF header at which the program header is present.

e_shoff: This member provides offset from the ELF header at which the section header is present.

e_flags: This member holds processor specific tags.

e_ehsize: This member holds the ELF header size in bytes.

e_phentsize: This member holds the size in bytes of one entry in the file's program header table.

e_phnum: This member holds

```
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                   ELF64
  Data:                   2's complement, little endian
  Version:                 1 (current)
  OS/ABI:                  UNIX - System V
  ABI Version:             0
  Type:                   DYN (Shared object file)
  Machine:                 Advanced Micro Devices X86-64
  Version:                 0x1
  Entry point address:     0x540
  Start of program headers: 64 (bytes into file)
  Start of section headers: 6448 (bytes into file)
  Flags:                   0x0
  Size of this header:     64 (bytes)
  Size of program headers: 56 (bytes)
  Number of program headers: 9
  Size of section headers: 64 (bytes)
  Number of section headers: 29
  Section header string table index: 28
```

Figure 1: ELF header

the number of entries in the program header table.

e_shentsize: This member holds a section header's size in bytes.

e_shnum: This member holds the number of entries in the section header table.

To read the ELF header of our object file, 'study', we will use a binary utility tool called *readelf*. The following is the command to read the ELF header of any object file:

```
readelf -h study
```

The output is seen in Figure 1.

As can be seen from the ELF header, the object file format is ELF64. The object file is Little Endian. The ELF file is built for an x86-64 bit machine.

There are two important pieces of information present in the ELF header. One is the ELF program header part and the other is the ELF section header part.

ELF sections

When a program is compiled, different things are generated after compilation. I have grouped these into raw groups as shown below:

- Binary executable code
- Code for initialised variables
- Code for uninitialised variables
- Constant strings

- Information about the variable and function names used in the program
 - Debugging information
- There could be other information but I will only discuss what has been mentioned here.

These groups are combined and they are provided specific names. Compilers generally have common names for these mentioned groups. All binary executable code goes into the *.text* section, all initialised variables are grouped in the *.data* section, all uninitialised variables are grouped in the *.bss* section, all constant strings are grouped in *.shstrtab* section, all variable and function names go in *.symtab* (which is called the symbol table) and all debug information is grouped in the *.debug* section. All this information is put in the section part of the ELF file. The location of the section header in an ELF file is provided by the *e_shoff* variable, which is present in the ELF header. *e_shoff* provides the offset at which the ELF section header is present in the ELF file. *e_shnum* is the number of section header entries in the ELF file and the size of each entry is *e_shentsize*.

Use the following command to see all the sections that are present in our *study.c* example:

```
readelf -S study
```